COMP102 Lecture 9: Graphs II: Algorithms

BFS, DFS, components, and shortest paths

Jalal Maaouni
May 31, 2026

UM6P University Mohammed VI Polytechnic
THE UM6P VANGUARD CENTER
#Empowering Minds.

Plan

1. Motivation
2. Recap adjacency lists
3. Traversal problem
4. BFS
5. Shortest path intuition
6. Path reconstruction
7. DFS
8. Connected components
9. Complexity
10. Dijkstra for weighted graphs

Motivation

From graph data to graph questions

From graph data to graph questions

Last time

We learned how to store a graph: vertices, edges, adjacency lists, matrices, directions, and weights.

From graph data to graph questions

Last time

We learned how to store a graph: vertices, edges, adjacency lists, matrices, directions, and weights.

Today

We use the graph to answer questions:

From graph data to graph questions

Last time

We learned how to store a graph: vertices, edges, adjacency lists, matrices, directions, and weights.

Today

We use the graph to answer questions:

- Which vertices can we reach?

From graph data to graph questions

Last time

We learned how to store a graph: vertices, edges, adjacency lists, matrices, directions, and weights.

Today

We use the graph to answer questions:

- Which vertices can we reach?
- What is the shortest number of steps?

From graph data to graph questions

Last time

We learned how to store a graph: vertices, edges, adjacency lists, matrices, directions, and weights.

Today

We use the graph to answer questions:

- Which vertices can we reach?
- What is the shortest number of steps?
- How many disconnected groups exist?

From graph data to graph questions

Last time

We learned how to store a graph: vertices, edges, adjacency lists, matrices, directions, and weights.

Today

We use the graph to answer questions:

- Which vertices can we reach?
- What is the shortest number of steps?
- How many disconnected groups exist?
- What changes when edges have weights?

From graph data to graph questions

Last time

We learned how to store a graph: vertices, edges, adjacency lists, matrices, directions, and weights.

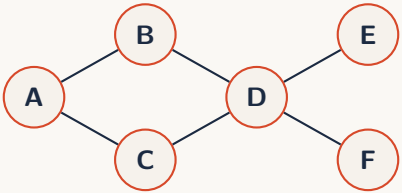
Today

We use the graph to answer questions:

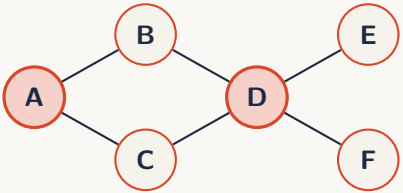
- Which vertices can we reach?
- What is the shortest number of steps?
- How many disconnected groups exist?
- What changes when edges have weights?

The core operation is traversal: start somewhere, move through edges, and remember what you have seen.

One graph, many questions



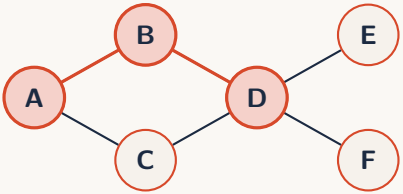
One graph, many questions



Reachability

Can we get from A to D?

One graph, many questions



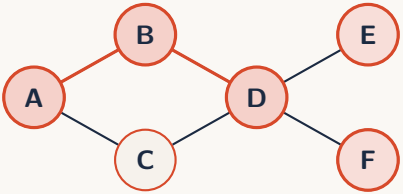
Reachability

Can we get from A to D?

Shortest path

What is the fewest number of edges?

One graph, many questions



Reachability

Can we get from A to D?

Shortest path

What is the fewest number of edges?

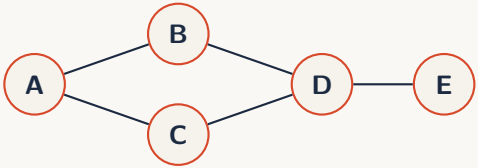
Expansion

What can we discover next?

Recap adjacency lists

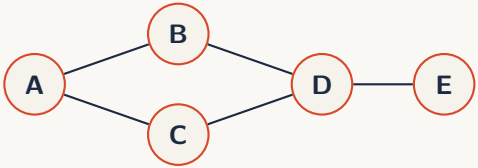
Adjacency list recap

```
graph = {
    "A": ["B", "C"],
    "B": ["A", "D"],
    "C": ["A", "D"],
    "D": ["B", "C", "E"],
    "E": ["D"],
}
```



Adjacency list recap

```python
graph = {
    "A": ["B", "C"],
    "B": ["A", "D"],
    "C": ["A", "D"],
    "D": ["B", "C", "E"],
    "E": ["D"],
}
```

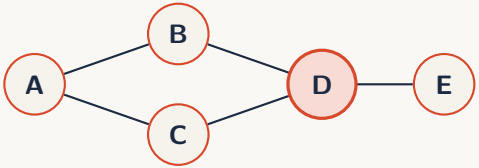


Meaning

`graph[u]` is the list of vertices directly reachable from u.

Adjacency list recap

```python
graph = {
    "A": ["B", "C"],
    "B": ["A", "D"],
    "C": ["A", "D"],
    "D": ["B", "C", "E"],
    "E": ["D"],
}
```



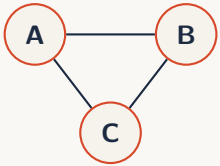
For D, the neighbors are B, C, and E.

Meaning

`graph[u]` is the list of vertices directly reachable from u.

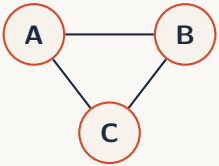
The one loop every graph algorithm starts with

```
for neighbor in graph[node]:
    # inspect this edge:
    # node -> neighbor
```



The one loop every graph algorithm starts with

```
for neighbor in graph[node]:
    # inspect this edge:
    # node -> neighbor
```

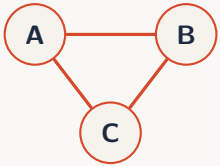


Traversal needs memory

When a graph has cycles, moving blindly is not enough. We need to know which vertices we already visited.

The one loop every graph algorithm starts with

```
for neighbor in graph[node]:
    # inspect this edge:
    # node -> neighbor
```



Traversal needs memory

When a graph has cycles, moving blindly is not enough. We need to know which vertices we already visited.

```
visited = set()
```

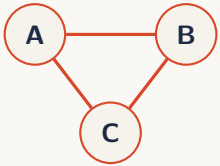
The one loop every graph algorithm starts with

```
for neighbor in graph[node]:
    # inspect this edge:
    # node -> neighbor
```

Traversal needs memory

When a graph has cycles, moving blindly is not enough. We need to know which vertices we already visited.

```
visited = set()
```



Without `visited`, a triangle can trap the algorithm forever.

Traversal problem

Traversal: visit everything reachable

Definition Graph traversal

A traversal algorithm starts from one vertex and systematically visits vertices reachable through edges.

Traversal: visit everything reachable

Definition Graph traversal

A traversal algorithm starts from one vertex and systematically visits vertices reachable through edges.

Input

Traversal: visit everything reachable

Definition Graph traversal

A traversal algorithm starts from one vertex and systematically visits vertices reachable through edges.

Input

- a graph;

Traversal: visit everything reachable

Definition Graph traversal

A traversal algorithm starts from one vertex and systematically visits vertices reachable through edges.

Input

- a graph;
- a start vertex;

Traversal: visit everything reachable

Definition Graph traversal

A traversal algorithm starts from one vertex and systematically visits vertices reachable through edges.

Input

- a graph;
- a start vertex;
- a rule for choosing what to explore next.

Traversal: visit everything reachable

Definition Graph traversal

A traversal algorithm starts from one vertex and systematically visits vertices reachable through edges.

Input

- a graph;
- a start vertex;
- a rule for choosing what to explore next.

Output

Depending on the problem:

Traversal: visit everything reachable

Definition Graph traversal

A traversal algorithm starts from one vertex and systematically visits vertices reachable through edges.

Input

- a graph;
- a start vertex;
- a rule for choosing what to explore next.

Output

Depending on the problem:

- visit order;

Traversal: visit everything reachable

Definition Graph traversal

A traversal algorithm starts from one vertex and systematically visits vertices reachable through edges.

Input

- a graph;
- a start vertex;
- a rule for choosing what to explore next.

Output

Depending on the problem:

- visit order;
- reachable set;

Traversal: visit everything reachable

Definition Graph traversal

A traversal algorithm starts from one vertex and systematically visits vertices reachable through edges.

Input

- a graph;
- a start vertex;
- a rule for choosing what to explore next.

Output

Depending on the problem:

- visit order;
- reachable set;
- distances;

Traversal: visit everything reachable

Definition Graph traversal

A traversal algorithm starts from one vertex and systematically visits vertices reachable through edges.

Input

- a graph;
- a start vertex;
- a rule for choosing what to explore next.

Output

Depending on the problem:

- visit order;
- reachable set;
- distances;
- parents;

Traversal: visit everything reachable

Definition Graph traversal

A traversal algorithm starts from one vertex and systematically visits vertices reachable through edges.

Input

- a graph;
- a start vertex;
- a rule for choosing what to explore next.

Output

Depending on the problem:

- visit order;
- reachable set;
- distances;
- parents;
- components.

The frontier: discovered but not explored

Definition Frontier

During traversal, the frontier contains vertices that have been discovered, but whose neighbors have not yet been explored.

The frontier: discovered but not explored

Definition Frontier

During traversal, the frontier contains vertices that have been discovered, but whose neighbors have not yet been explored.

Three kinds of vertices

The frontier: discovered but not explored

Definition Frontier

During traversal, the frontier contains vertices that have been discovered, but whose neighbors have not yet been explored.

Three kinds of vertices

- **unseen**: not discovered yet;

The frontier: discovered but not explored

Definition Frontier

During traversal, the frontier contains vertices that have been discovered, but whose neighbors have not yet been explored.

Three kinds of vertices

- **unseen**: not discovered yet;
- **frontier**: discovered, waiting;

The frontier: discovered but not explored

Definition Frontier

During traversal, the frontier contains vertices that have been discovered, but whose neighbors have not yet been explored.

Three kinds of vertices

- **unseen**: not discovered yet;
- **frontier**: discovered, waiting;
- **done**: explored completely.

The frontier: discovered but not explored

Definition Frontier

During traversal, the frontier contains vertices that have been discovered, but whose neighbors have not yet been explored.

Three kinds of vertices

- **unseen**: not discovered yet;
- **frontier**: discovered, waiting;
- **done**: explored completely.

The whole strategy

The frontier: discovered but not explored

Definition Frontier

During traversal, the frontier contains vertices that have been discovered, but whose neighbors have not yet been explored.

Three kinds of vertices

- **unseen**: not discovered yet;
- **frontier**: discovered, waiting;
- **done**: explored completely.

The whole strategy

1. start with one vertex;

The frontier: discovered but not explored

Definition Frontier

During traversal, the frontier contains vertices that have been discovered, but whose neighbors have not yet been explored.

Three kinds of vertices

- **unseen**: not discovered yet;
- **frontier**: discovered, waiting;
- **done**: explored completely.

The whole strategy

1. start with one vertex;
2. put it in the frontier;

The frontier: discovered but not explored

Definition Frontier

During traversal, the frontier contains vertices that have been discovered, but whose neighbors have not yet been explored.

Three kinds of vertices

- **unseen**: not discovered yet;
- **frontier**: discovered, waiting;
- **done**: explored completely.

The whole strategy

1. start with one vertex;
2. put it in the frontier;
3. repeatedly choose one frontier vertex;

The frontier: discovered but not explored

Definition Frontier

During traversal, the frontier contains vertices that have been discovered, but whose neighbors have not yet been explored.

Three kinds of vertices

- **unseen**: not discovered yet;
- **frontier**: discovered, waiting;
- **done**: explored completely.

The whole strategy

1. start with one vertex;
2. put it in the frontier;
3. repeatedly choose one frontier vertex;
4. discover its unseen neighbors.

The frontier: discovered but not explored

Definition Frontier

During traversal, the frontier contains vertices that have been discovered, but whose neighbors have not yet been explored.

Three kinds of vertices

- **unseen**: not discovered yet;
- **frontier**: discovered, waiting;
- **done**: explored completely.

The whole strategy

1. start with one vertex;
2. put it in the frontier;
3. repeatedly choose one frontier vertex;
4. discover its unseen neighbors.

Different traversal algorithms mainly differ by how they choose the next vertex from the frontier.

Two meaningful ways to choose from the frontier

Two meaningful ways to choose from the frontier

Strategy 1: oldest first

Choose the vertex that has been waiting the longest.

Two meaningful ways to choose from the frontier

Strategy 1: oldest first

Choose the vertex that has been waiting the longest.

- explore near vertices before far vertices;

Two meaningful ways to choose from the frontier

Strategy 1: oldest first

Choose the vertex that has been waiting the longest.

- explore near vertices before far vertices;
- grow the search in layers;

Two meaningful ways to choose from the frontier

Strategy 1: oldest first

Choose the vertex that has been waiting the longest.

- explore near vertices before far vertices;
- grow the search in layers;
- preserve distance from the start.

Two meaningful ways to choose from the frontier

Strategy 1: oldest first

Choose the vertex that has been waiting the longest.

- explore near vertices before far vertices;
- grow the search in layers;
- preserve distance from the start.

Queue

Two meaningful ways to choose from the frontier

Strategy 1: oldest first

Choose the vertex that has been waiting the longest.

- explore near vertices before far vertices;
- grow the search in layers;
- preserve distance from the start.

Queue

Strategy 2: newest first

Choose the vertex discovered most recently.

Two meaningful ways to choose from the frontier

Strategy 1: oldest first

Choose the vertex that has been waiting the longest.

- explore near vertices before far vertices;
- grow the search in layers;
- preserve distance from the start.

Queue

Strategy 2: newest first

Choose the vertex discovered most recently.

- follow one branch deeply;

Two meaningful ways to choose from the frontier

Strategy 1: oldest first

Choose the vertex that has been waiting the longest.

- explore near vertices before far vertices;
- grow the search in layers;
- preserve distance from the start.

Queue

Strategy 2: newest first

Choose the vertex discovered most recently.

- follow one branch deeply;
- backtrack when stuck;

Two meaningful ways to choose from the frontier

Strategy 1: oldest first

Choose the vertex that has been waiting the longest.

- explore near vertices before far vertices;
- grow the search in layers;
- preserve distance from the start.

Queue

Strategy 2: newest first

Choose the vertex discovered most recently.

- follow one branch deeply;
- backtrack when stuck;
- useful for structural exploration.

Two meaningful ways to choose from the frontier

Strategy 1: oldest first

Choose the vertex that has been waiting the longest.

- explore near vertices before far vertices;
- grow the search in layers;
- preserve distance from the start.

Queue

Strategy 2: newest first

Choose the vertex discovered most recently.

- follow one branch deeply;
- backtrack when stuck;
- useful for structural exploration.

Stack / recursion

These strategies have names

These strategies have names

Breadth-First Search

BFS uses the oldest discovered vertex first.

These strategies have names

Breadth-First Search

BFS uses the oldest discovered vertex first.

- data structure: queue;

These strategies have names

Breadth-First Search

BFS uses the oldest discovered vertex first.

- data structure: queue;
- explores by layers;

These strategies have names

Breadth-First Search

BFS uses the oldest discovered vertex first.

- data structure: queue;
- explores by layers;
- gives shortest paths in unweighted graphs.

These strategies have names

Breadth-First Search

BFS uses the oldest discovered vertex first.

- data structure: queue;
- explores by layers;
- gives shortest paths in unweighted graphs.

Depth-First Search

DFS uses the newest discovered vertex first.

These strategies have names

Breadth-First Search

BFS uses the oldest discovered vertex first.

- data structure: queue;
- explores by layers;
- gives shortest paths in unweighted graphs.

Depth-First Search

DFS uses the newest discovered vertex first.

- data structure: stack or recursion;

These strategies have names

Breadth-First Search

BFS uses the oldest discovered vertex first.

- data structure: queue;
- explores by layers;
- gives shortest paths in unweighted graphs.

Depth-First Search

DFS uses the newest discovered vertex first.

- data structure: stack or recursion;
- explores deeply before backtracking;

These strategies have names

Breadth-First Search

BFS uses the oldest discovered vertex first.

- data structure: queue;
- explores by layers;
- gives shortest paths in unweighted graphs.

Depth-First Search

DFS uses the newest discovered vertex first.

- data structure: stack or recursion;
- explores deeply before backtracking;
- useful for components, cycles, and advanced graph structure.

These strategies have names

Breadth-First Search

BFS uses the oldest discovered vertex first.

- data structure: queue;
- explores by layers;
- gives shortest paths in unweighted graphs.

Depth-First Search

DFS uses the newest discovered vertex first.

- data structure: stack or recursion;
- explores deeply before backtracking;
- useful for components, cycles, and advanced graph structure.

BFS and DFS are not arbitrary algorithms. They are the two simplest fundamental ways to manage the traversal frontier.

BFS

BFS: oldest discovered vertex first

Definition Breadth-First Search

Breadth-First Search is a traversal strategy that always explores the oldest discovered vertex that has not yet been processed.

BFS: oldest discovered vertex first

Definition Breadth-First Search

Breadth-First Search is a traversal strategy that always explores the oldest discovered vertex that has not yet been processed.

Frontier rule

BFS: oldest discovered vertex first

Definition Breadth-First Search

Breadth-First Search is a traversal strategy that always explores the oldest discovered vertex that has not yet been processed.

Frontier rule

- discovered vertices wait in a queue;

BFS: oldest discovered vertex first

Definition Breadth-First Search

Breadth-First Search is a traversal strategy that always explores the oldest discovered vertex that has not yet been processed.

Frontier rule

- discovered vertices wait in a queue;
- pop from the front;

BFS: oldest discovered vertex first

Definition Breadth-First Search

Breadth-First Search is a traversal strategy that always explores the oldest discovered vertex that has not yet been processed.

Frontier rule

- discovered vertices wait in a queue;
- pop from the front;
- insert new unseen neighbors at the back.

BFS: oldest discovered vertex first

Definition Breadth-First Search

Breadth-First Search is a traversal strategy that always explores the oldest discovered vertex that has not yet been processed.

Frontier rule

- discovered vertices wait in a queue;
- pop from the front;
- insert new unseen neighbors at the back.

BFS: oldest discovered vertex first

Definition Breadth-First Search

Breadth-First Search is a traversal strategy that always explores the oldest discovered vertex that has not yet been processed.

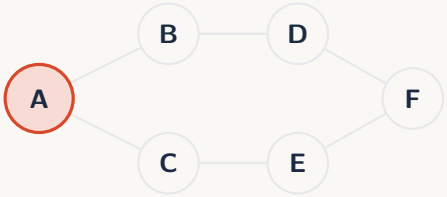
Frontier rule

- discovered vertices wait in a queue;
- pop from the front;
- insert new unseen neighbors at the back.

Queue discipline

First discovered, first explored.

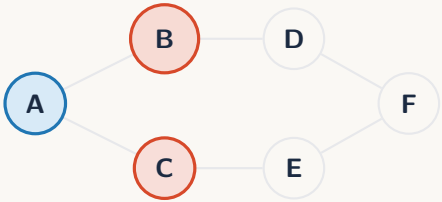
BFS traversal example



Traversal order

A

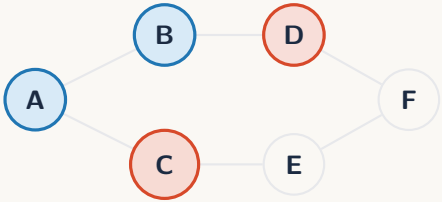
BFS traversal example



Traversal order

A, B

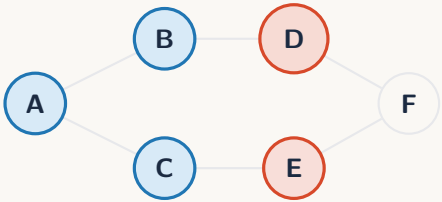
BFS traversal example



Traversal order

A, B, C

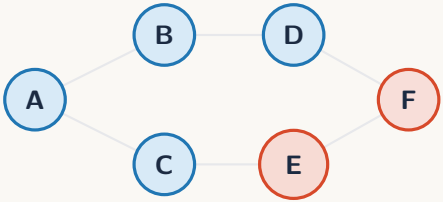
BFS traversal example



Traversal order

A, B, C, D

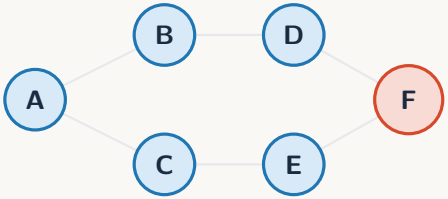
BFS traversal example



Traversal order

A, B, C, D, E

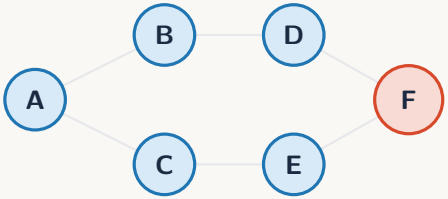
BFS traversal example



Traversal order

A, B, C, D, E, F

BFS traversal example



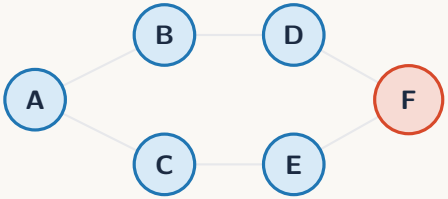
Traversal order

A, B, C, D, E, F

Key observation

The algorithm does not jump randomly. The queue controls the order of exploration.

BFS traversal example



Traversal order

A, B, C, D, E, F

Key observation

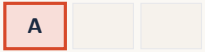
The algorithm does not jump randomly. The queue controls the order of exploration.

This is the BFS traversal of the graph starting from A.

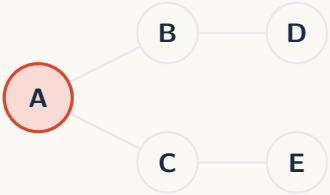
BFS operation: pop from the queue

Current state

Before exploring neighbors, BFS removes the front element of the queue.



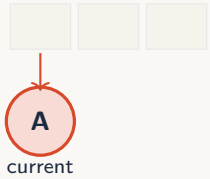
front

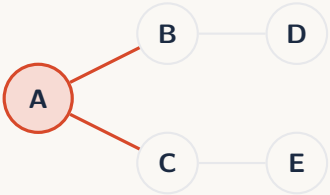


BFS operation: pop from the queue

Current state

Before exploring neighbors, BFS removes the front element of the queue.



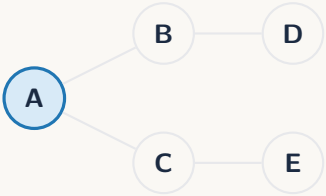


Popping selects the next vertex to process. It does not yet discover anything new.

BFS operation: insert unseen neighbors

After processing A

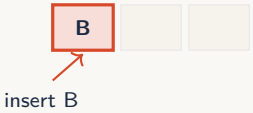
A has two unseen neighbors: B and C.

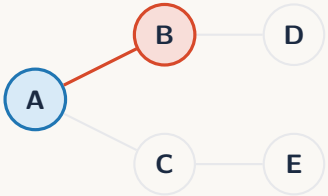


BFS operation: insert unseen neighbors

After processing A

A has two unseen neighbors: B and C.

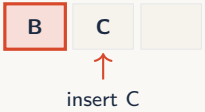


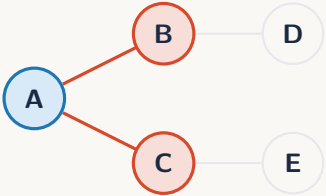


BFS operation: insert unseen neighbors

After processing A

A has two unseen neighbors: B and C.



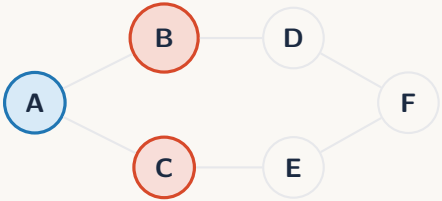


Inserted vertices are discovered, but they are not explored immediately. They wait their turn.

BFS operation: repeat the same rule

Queue before next step

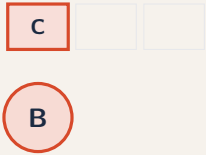
B	C	

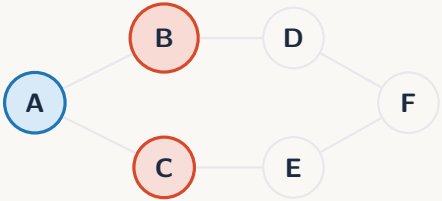


BFS operation: repeat the same rule

Queue before next step

Pop B





BFS operation: repeat the same rule

Queue before next step

B C

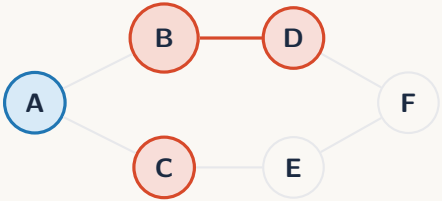
Pop B

C

B

Insert B's unseen neighbor

C D

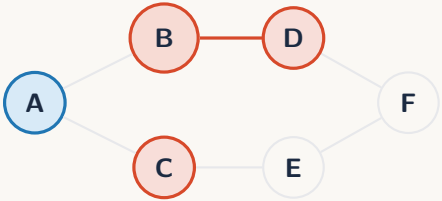


BFS operation: repeat the same rule

Queue before next step

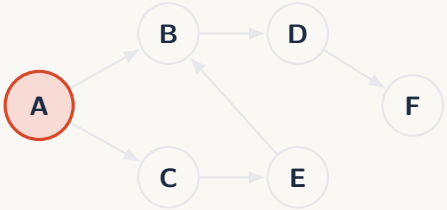
Pop B

Insert B's unseen neighbor



Same rule every time: pop one vertex, inspect its neighbors, insert only unseen ones.

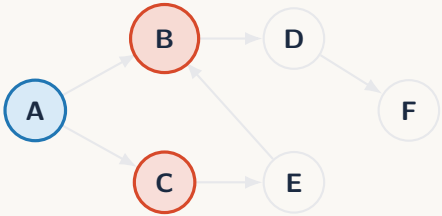
BFS on a directed graph



Directed rule

From a vertex, BFS only follows outgoing edges.

BFS on a directed graph



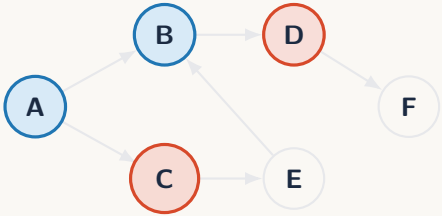
Directed rule

From a vertex, BFS only follows outgoing edges.

Traversal order

A, B

BFS on a directed graph



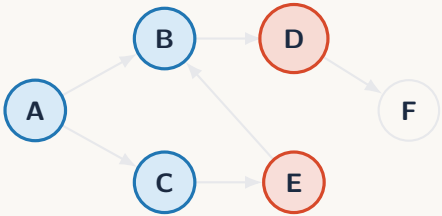
Directed rule

From a vertex, BFS only follows outgoing edges.

Traversal order

A, B, C

BFS on a directed graph



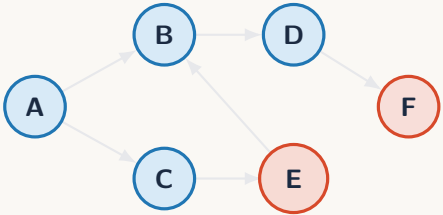
Directed rule

From a vertex, BFS only follows outgoing edges.

Traversal order

A, B, C, D

BFS on a directed graph



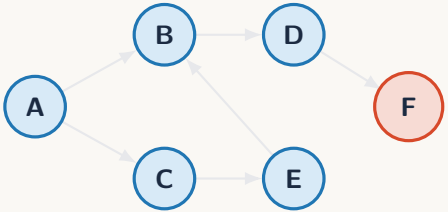
Directed rule

From a vertex, BFS only follows outgoing edges.

Traversal order

A, B, C, D, E

BFS on a directed graph



Directed rule

From a vertex, BFS only follows outgoing edges.

Traversal order

A, B, C, D, E, F

The same queue rule works. Only the meaning of "neighbor" changes: in a directed graph, neighbors are outgoing neighbors.

BFS code

```
1  from collections import deque
2
3  def bfs(graph, start):
4      visited = {start}
5      queue = deque([start])
6
7      while queue:
8          node = queue.popleft()
9          print(node)
10
11         for neighbor in graph[node]:
12             if neighbor not in visited:
13                 visited.add(neighbor)
14                 queue.append(neighbor)
```

BFS code

```python
from collections import deque

def bfs(graph, start):
    visited = {start}
    queue = deque([start])

    while queue:
        node = queue.popleft()
        print(node)

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
```

The code follows exactly the animation: pop one vertex, inspect its neighbors, insert unseen neighbors.

Shortest path intuition

Now BFS has an important consequence

What we already know

BFS explores vertices using a queue: oldest discovered vertex first.

Now BFS has an important consequence

What we already know

BFS explores vertices using a queue: oldest discovered vertex first.

Consequence in unweighted graphs

Because all edges cost one step, this queue order processes vertices by increasing number of edges from the start.

Now BFS has an important consequence

What we already know

BFS explores vertices using a queue: oldest discovered vertex first.

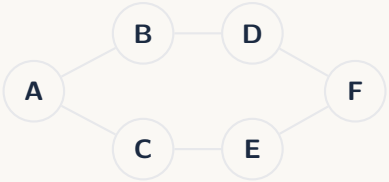
Consequence in unweighted graphs

Because all edges cost one step, this queue order processes vertices by increasing number of edges from the start.

So BFS is first a traversal algorithm. Shortest paths are a consequence of that traversal order.

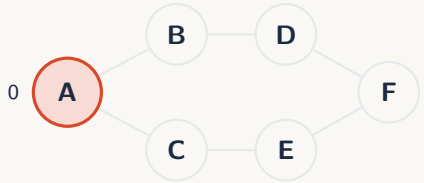
BFS distances in unweighted graphs

```python
def bfs_distances(graph, start):
    distance = {start: 0}
    queue = deque([start])

    while queue:
        node = queue.popleft()

        for neighbor in graph[node]:
            if neighbor not in distance:
                distance[neighbor] = distance[node] + 1
                queue.append(neighbor)

    return distance
```



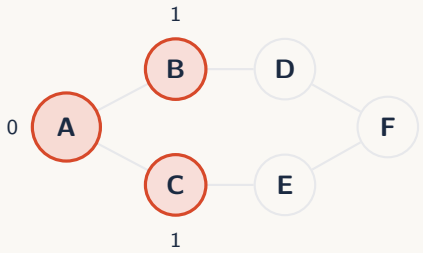
BFS distances in unweighted graphs

```python
def bfs_distances(graph, start):
    distance = {start: 0}
    queue = deque([start])

    while queue:
        node = queue.popleft()

        for neighbor in graph[node]:
            if neighbor not in distance:
                distance[neighbor] =
    distance[node] + 1
                queue.append(neighbor)

    return distance
```



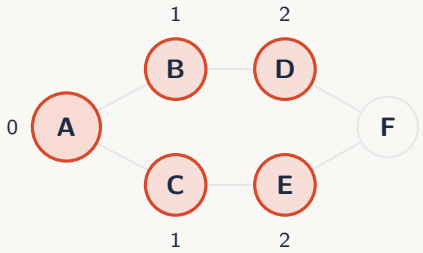
BFS distances in unweighted graphs

```python
def bfs_distances(graph, start):
    distance = {start: 0}
    queue = deque([start])

    while queue:
        node = queue.popleft()

        for neighbor in graph[node]:
            if neighbor not in distance:
                distance[neighbor] = distance[node] + 1
                queue.append(neighbor)

    return distance
```



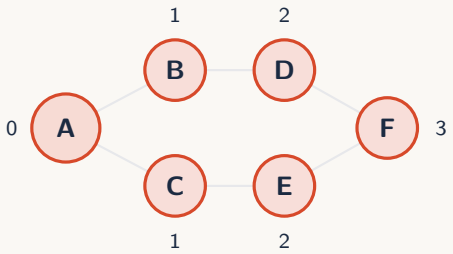
BFS distances in unweighted graphs

```python
def bfs_distances(graph, start):
    distance = {start: 0}
    queue = deque([start])

    while queue:
        node = queue.popleft()

        for neighbor in graph[node]:
            if neighbor not in distance:
                distance[neighbor] =
    distance[node] + 1
                queue.append(neighbor)

    return distance
```



BFS distances in unweighted graphs

```python
def bfs_distances(graph, start):
    distance = {start: 0}
    queue = deque([start])

    while queue:
        node = queue.popleft()

        for neighbor in graph[node]:
            if neighbor not in distance:
                distance[neighbor] =
    distance[node] + 1
                queue.append(neighbor)

    return distance
```



The first time BFS discovers a vertex, it has found the minimum number of edges from the start.

Path reconstruction

Distances are not enough: store parents

```
def shortest_path(graph, start, target):
    parent = {start: None}
    queue = deque([start])

    while queue:
        node = queue.popleft()
        if node == target:
            break

        for neighbor in graph[node]:
            if neighbor not in parent:
                parent[neighbor] = node
                queue.append(neighbor)
```

Distances are not enough: store parents

```python
def shortest_path(graph, start, target):
    parent = {start: None}
    queue = deque([start])

    while queue:
        node = queue.popleft()
        if node == target:
            break

        for neighbor in graph[node]:
            if neighbor not in parent:
                parent[neighbor] = node
                queue.append(neighbor)
```

Parent meaning

```python
parent[neighbor] = node
```

means: we reached `neighbor` from `node`.

Distances are not enough: store parents

```
def shortest_path(graph, start, target):
    parent = {start: None}
    queue = deque([start])

    while queue:
        node = queue.popleft()
        if node == target:
            break

        for neighbor in graph[node]:
            if neighbor not in parent:
                parent[neighbor] = node
                queue.append(neighbor)
```

Parent meaning

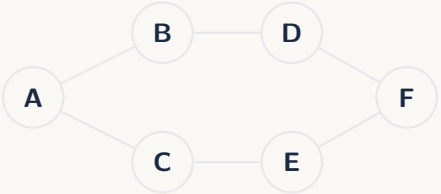
```
parent[neighbor] = node
```

means: we reached `neighbor` from `node`.

The parent dictionary stores a shortest-path tree rooted at the start vertex.

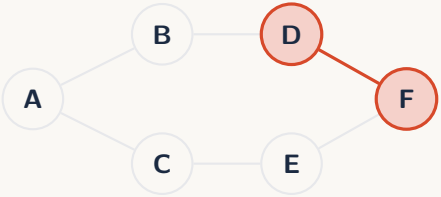
Reconstruct the path backward

```
if target not in parent:
    return None

path = []
current = target

while current is not None:
    path.append(current)
    current = parent[current]

path.reverse()
return path
```



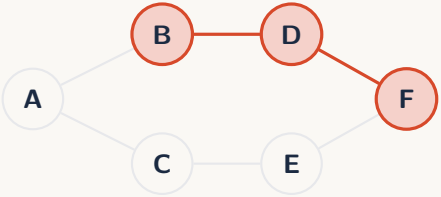
Reconstruct the path backward

```
if target not in parent:
    return None

path = []
current = target

while current is not None:
    path.append(current)
    current = parent[current]

path.reverse()
return path
```



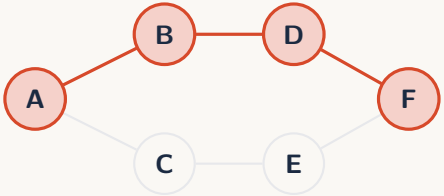
Reconstruct the path backward

```python
if target not in parent:
    return None

path = []
current = target

while current is not None:
    path.append(current)
    current = parent[current]

path.reverse()
return path
```



Reconstruct the path backward

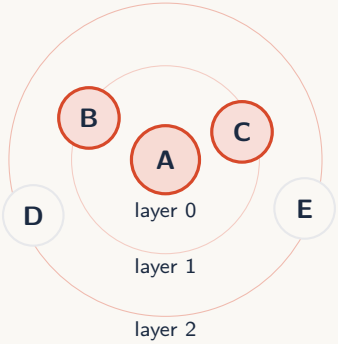
```
if target not in parent:
    return None

path = []
current = target

while current is not None:
    path.append(current)
    current = parent[current]

path.reverse()
return path
```



Backward then reverse

$F \leftarrow D \leftarrow B \leftarrow A$ becomes $A \rightarrow B \rightarrow D \rightarrow F$.

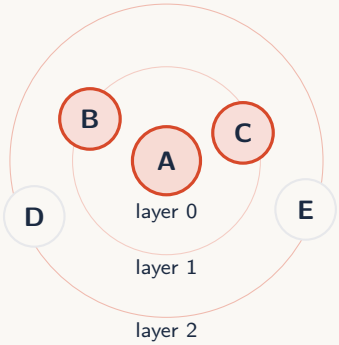
Why BFS gives shortest paths here



Why BFS gives shortest paths here

Layer invariant

Before BFS starts processing distance $k+1$, it has already processed every vertex at distance k.



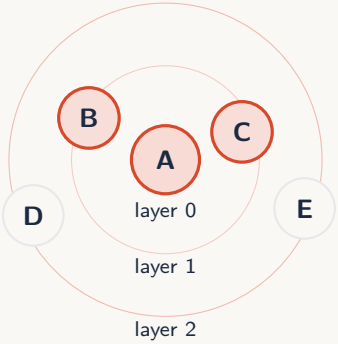
Why BFS gives shortest paths here

Layer invariant

Before BFS starts processing distance $k+1$, it has already processed every vertex at distance k.

Consequence

If a target appears for the first time at layer k, no path with fewer than k edges exists.



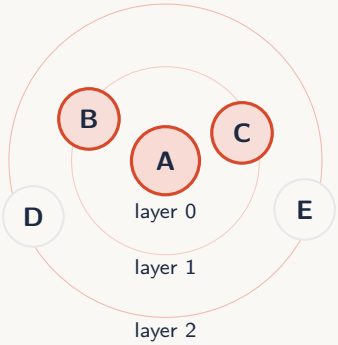
Why BFS gives shortest paths here

Layer invariant

Before BFS starts processing distance $k+1$, it has already processed every vertex at distance k.

Consequence

If a target appears for the first time at layer k, no path with fewer than k edges exists.



This is only about number of edges. It assumes every edge has equal cost.

DFS

DFS: newest discovered vertex first

Definition Depth-First Search

Depth-First Search is a traversal strategy that always explores the newest discovered vertex that has not yet been processed.

DFS: newest discovered vertex first

Definition Depth-First Search

Depth-First Search is a traversal strategy that always explores the newest discovered vertex that has not yet been processed.

Frontier rule

DFS: newest discovered vertex first

Definition Depth-First Search

Depth-First Search is a traversal strategy that always explores the newest discovered vertex that has not yet been processed.

Frontier rule

- discovered vertices wait in a stack;

DFS: newest discovered vertex first

Definition Depth-First Search

Depth-First Search is a traversal strategy that always explores the newest discovered vertex that has not yet been processed.

Frontier rule

- discovered vertices wait in a stack;
- pop from the top;

DFS: newest discovered vertex first

Definition Depth-First Search

Depth-First Search is a traversal strategy that always explores the newest discovered vertex that has not yet been processed.

Frontier rule

- discovered vertices wait in a stack;
- pop from the top;
- insert new unseen neighbors on top;

DFS: newest discovered vertex first

Definition Depth-First Search

Depth-First Search is a traversal strategy that always explores the newest discovered vertex that has not yet been processed.

Frontier rule

- discovered vertices wait in a stack;
- pop from the top;
- insert new unseen neighbors on top;
- the newest vertex is explored next.

DFS: newest discovered vertex first

Definition Depth-First Search

Depth-First Search is a traversal strategy that always explores the newest discovered vertex that has not yet been processed.

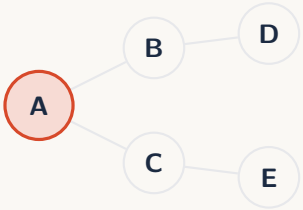
Frontier rule

- discovered vertices wait in a stack;
- pop from the top;
- insert new unseen neighbors on top;
- the newest vertex is explored next.

Stack discipline

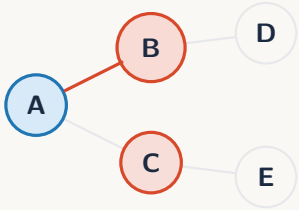
Last discovered, first explored.

DFS traversal example



Traversal order
A

DFS traversal example



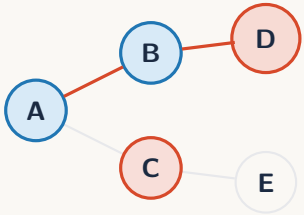
Traversal order

A, B

Key observation

DFS commits to one branch before returning to older alternatives.

DFS traversal example



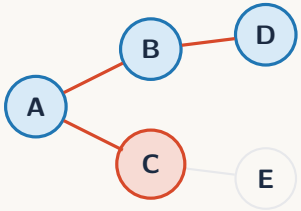
Traversal order

A, B, D

Key observation

DFS commits to one branch before returning to older alternatives.

DFS traversal example



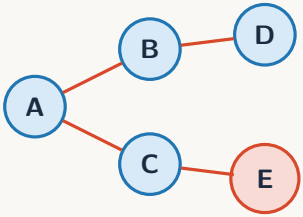
Traversal order

A, B, D, C

Key observation

DFS commits to one branch before returning to older alternatives.

DFS traversal example



Traversal order

A, B, D, C, E

Key observation

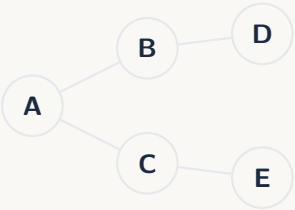
DFS commits to one branch before returning to older alternatives.

This is the DFS traversal of the graph starting from A.

DFS operation: push the start vertex

Initial state

DFS starts by putting the start vertex on the stack.

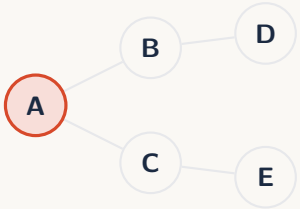


DFS operation: push the start vertex

Initial state

DFS starts by putting the start vertex on the stack.

top



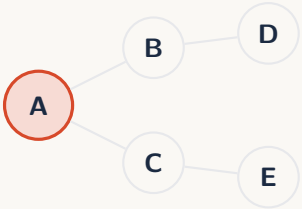
The stack contains vertices that have been discovered but not yet processed.

DFS operation: pop from the stack

Current stack

The top of the stack is the next vertex to process.

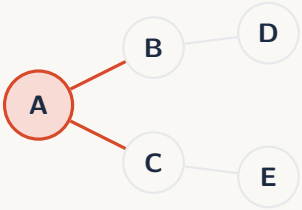
top



DFS operation: pop from the stack

Current stack

The top of the stack is the next vertex to process.

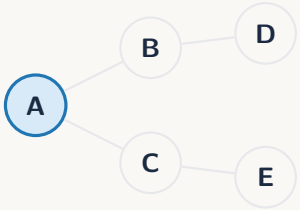


Popping chooses the next vertex. After that, we inspect its neighbors.

DFS operation: push unseen neighbors

After processing A

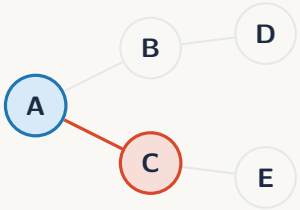
A has two unseen neighbors: B and C.



DFS operation: push unseen neighbors

After processing A

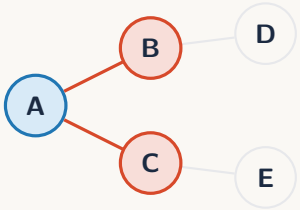
A has two unseen neighbors: B and C.



DFS operation: push unseen neighbors

After processing A

A has two unseen neighbors: B and C.



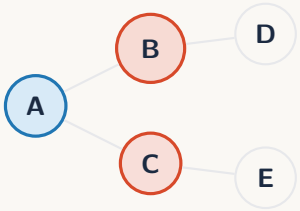
To visit B before C with a stack, we push C first, then B. The last pushed vertex is popped first.

DFS operation: repeat the same rule

Stack before next step

top

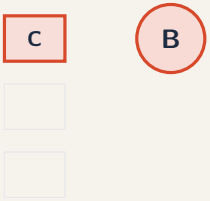
C

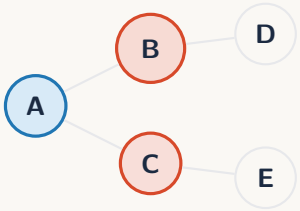


DFS operation: repeat the same rule

Stack before next step

Pop B

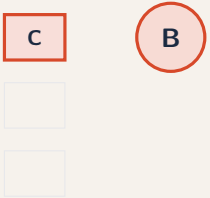


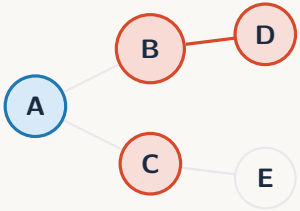


DFS operation: repeat the same rule

Stack before next step

Pop B



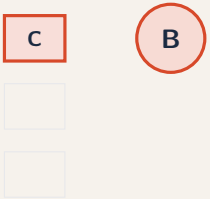


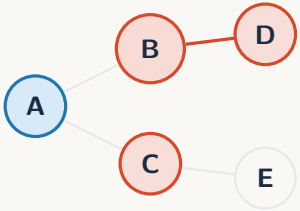
Push B's unseen neighbor

DFS operation: repeat the same rule

Stack before next step

Pop B





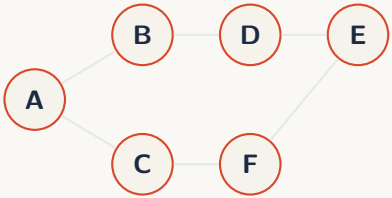
Push B's unseen neighbor

The stack makes DFS continue deeper before returning to C.

DFS is not organized by distance

DFS behavior

DFS follows the newest available vertex.



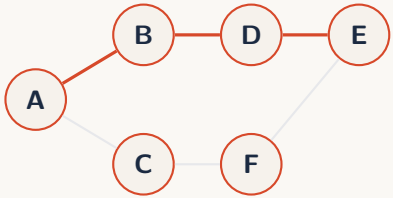
DFS is not organized by distance

DFS behavior

DFS follows the newest available vertex.

Consequence

It may reach a far vertex before a closer one.



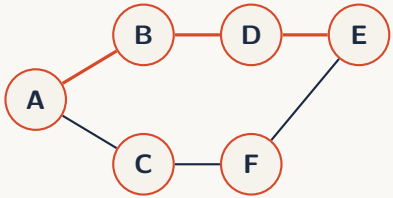
DFS is not organized by distance

DFS behavior

DFS follows the newest available vertex.

Consequence

It may reach a far vertex before a closer one.



Point

DFS may find one path to E, but not necessarily the shortest one first.

DFS is not organized by distance

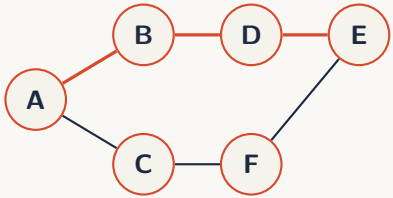
DFS behavior

DFS follows the newest available vertex.

Consequence

It may reach a far vertex before a closer one.

DFS is useful for exploration and structure, but it does not guarantee shortest paths.



Point

DFS may find one path to E, but not necessarily the shortest one first.

DFS code: recursive and iterative

Recursive DFS

```
def dfs(graph, node, visited):
    visited.add(node)
    print(node)

    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs(graph, neighbor,
    visited)
```

Iterative DFS

```
def dfs_iterative(graph, start):
    visited = set()
    stack = [start]
    while stack:
        node = stack.pop()
        if node in visited:
            continue
        visited.add(node)
        print(node)
        for neighbor in
    reversed(graph[node]):
            if neighbor not in
    visited:
                stack.append(neighbor)
```

DFS code: recursive and iterative

Recursive DFS

```python
def dfs(graph, node, visited):
    visited.add(node)
    print(node)

    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs(graph, neighbor,
    visited)
```

Iterative DFS

```python
def dfs_iterative(graph, start):
    visited = set()
    stack = [start]
    while stack:
        node = stack.pop()
        if node in visited:
            continue
        visited.add(node)
        print(node)
        for neighbor in
    reversed(graph[node]):
            if neighbor not in
    visited:
                stack.append(neighbor)
```

Recursive DFS is short, but on very deep graphs it can hit Python's recursion limit. Iterative DFS avoids that

BFS vs DFS

Question	BFS	DFS
Data structure	Queue	Stack / recursion
Exploration style	Layer by layer	Deep then backtrack
Unweighted shortest path	Yes	No
Memory behavior	Stores frontier	Stores current branch / stack
Common use	distances, levels	exhaustive search, components

BFS vs DFS

Question	BFS	DFS
Data structure	Queue	Stack / recursion
Exploration style	Layer by layer	Deep then backtrack
Unweighted shortest path	Yes	No
Memory behavior	Stores frontier	Stores current branch / stack
Common use	distances, levels	exhaustive search, components

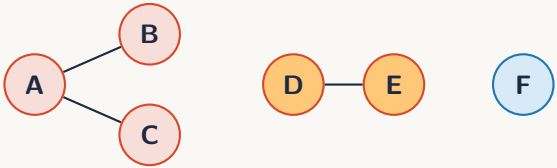
For this course, use BFS when distance matters. Use DFS when reachability or full exploration is enough.

Connected components

Connected components in undirected graphs

Definition Connected component

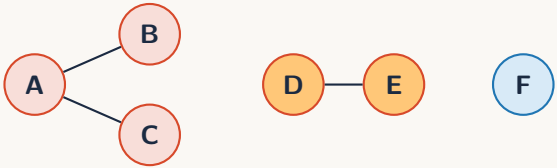
In an undirected graph, a connected component is a maximal group of vertices where every vertex can reach every other vertex in the group.



Connected components in undirected graphs

Definition Connected component

In an undirected graph, a connected component is a maximal group of vertices where every vertex can reach every other vertex in the group.



This graph has three connected components: $\{A, B, C\}$, $\{D, E\}$, and $\{F\}$.

Finding components: repeated traversal

```python
def collect_component(graph, start, visited):
    component = []
    stack = [start]
    while stack:
        node = stack.pop()
        if node in visited:
            continue
        visited.add(node)
        component.append(node)
        for neighbor in graph[node]:
            stack.append(neighbor)
    return component
```

```python
def connected_components(graph):
    visited = set()
    components = []
    for node in graph:
        if node not in visited:
            component = collect_component(
                graph, node, visited
            )

    components.append(component)
    return components
```

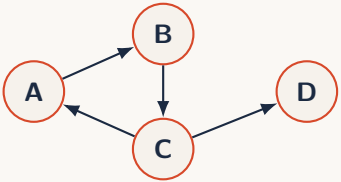
Finding components: repeated traversal

```python
def collect_component(graph, start, visited):
    component = []
    stack = [start]
    while stack:
        node = stack.pop()
        if node in visited:
            continue
        visited.add(node)
        component.append(node)
        for neighbor in graph[node]:
            stack.append(neighbor)
    return component
```

```python
def connected_components(graph):
    visited = set()
    components = []
    for node in graph:
        if node not in visited:
            component = collect_component(
                graph, node, visited
            )

    components.append(component)
    return components
```

For this course, connected components means undirected connected components unless stated otherwise.

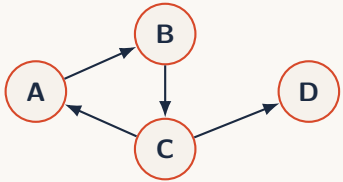
Directed graphs: weak vs strong components



Directed graphs: weak vs strong components

Weakly connected

Ignore arrow directions. If the underlying undirected graph is connected, the directed graph is weakly connected.



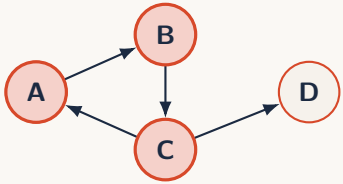
Directed graphs: weak vs strong components

Weakly connected

Ignore arrow directions. If the underlying undirected graph is connected, the directed graph is weakly connected.

Strongly connected

Respect arrow directions. Every vertex must be able to reach every other vertex.



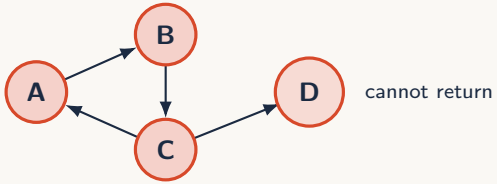
Directed graphs: weak vs strong components

Weakly connected

Ignore arrow directions. If the underlying undirected graph is connected, the directed graph is weakly connected.

Strongly connected

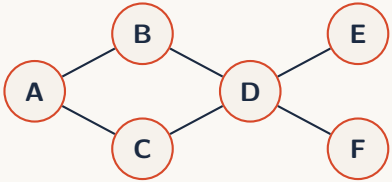
Respect arrow directions. Every vertex must be able to reach every other vertex.



A, B, C form a strongly connected component. D is reached from them, but cannot reach them back.

Complexity

Why BFS and DFS are $O(V + E)$

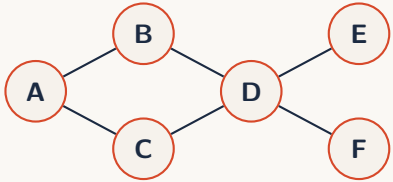


Why BFS and DFS are $O(V + E)$

Vertices

Each vertex is marked visited once.

$$O(V)$$



Why BFS and DFS are $O(V + E)$

Vertices

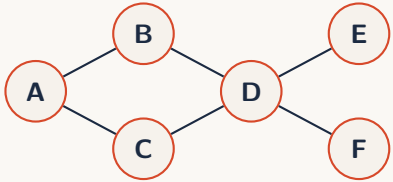
Each vertex is marked visited once.

$$O(V)$$

Edges

Across the whole algorithm, adjacency lists are scanned once per visited vertex.

$$O(E)$$



Complexity summary

Algorithm	Time	Space	Main extra data
BFS traversal	$O(V+E)$	$O(V)$	queue, visited
BFS distances	$O(V+E)$	$O(V)$	queue, distance
BFS path	$O(V+E)$	$O(V)$	queue, parent
DFS traversal	$O(V+E)$	$O(V)$	stack/recursion, visited
Components	$O(V+E)$	$O(V)$	visited, component list

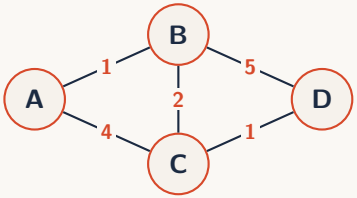
Complexity summary

Algorithm	Time	Space	Main extra data
BFS traversal	$O(V + E)$	$O(V)$	queue, visited
BFS distances	$O(V + E)$	$O(V)$	queue, distance
BFS path	$O(V + E)$	$O(V)$	queue, parent
DFS traversal	$O(V + E)$	$O(V)$	stack/recursion, visited
Components	$O(V + E)$	$O(V)$	visited, component list

For adjacency matrices, scanning neighbors costs more: checking all possible neighbors of a node costs $O(V)$.

Dijkstra for weighted graphs

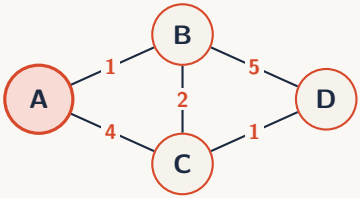
Weighted shortest path: Dijkstra's idea



Weighted shortest path: Dijkstra's idea

Problem

Find the minimum total cost from a start vertex to every other vertex in a graph with non-negative edge weights.



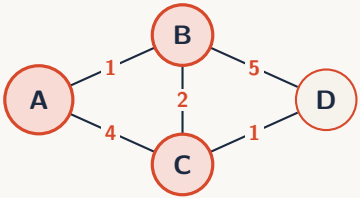
Weighted shortest path: Dijkstra's idea

Problem

Find the minimum total cost from a start vertex to every other vertex in a graph with non-negative edge weights.

Main idea

Always expand the unsettled vertex with the smallest known distance so far.



Weighted shortest path: Dijkstra's idea

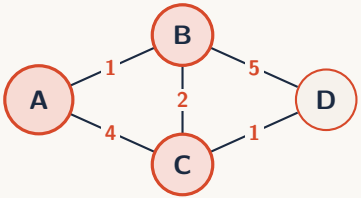
Problem

Find the minimum total cost from a start vertex to every other vertex in a graph with non-negative edge weights.

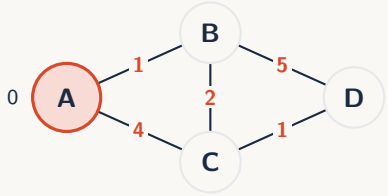
Main idea

Always expand the unsettled vertex with the smallest known distance so far.

Dijkstra is BFS with a priority queue instead of a normal queue.

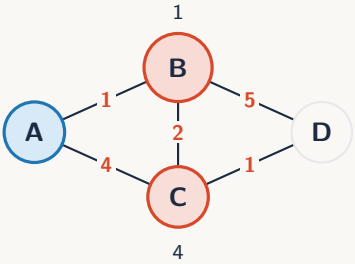


Dijkstra animation: choose smallest tentative distance



Priority queue contents
(0,A)

Dijkstra animation: choose smallest tentative distance



Priority queue contents

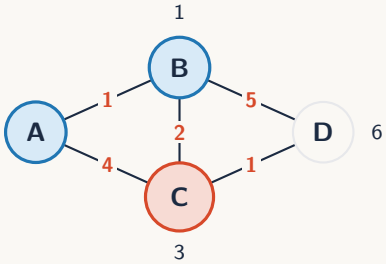
(1,B) (4,C)

Relaxation

When going through edge $u \to v$ with weight w, try:

$$\texttt{dist[v]} = \min(\texttt{dist[v]}, \texttt{dist[u]} + w)$$

Dijkstra animation: choose smallest tentative distance



Priority queue contents

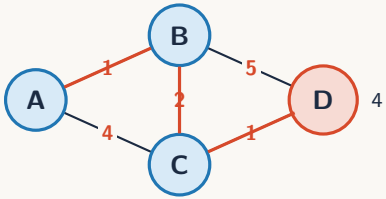
(3,C) (6,D)

Relaxation

When going through edge $u \to v$ with weight w, try:

$$\texttt{dist[v]} = \min(\texttt{dist[v]}, \texttt{dist[u]} + w)$$

Dijkstra animation: choose smallest tentative distance



Priority queue contents

(4,D)

Relaxation

When going through edge $u \to v$ with weight w, try:

$$\texttt{dist[v]} = \min(\texttt{dist[v]}, \texttt{dist[u]} + w)$$

Dijkstra code skeleton

```python
import heapq

def dijkstra(graph, start):
    distance = {start: 0}
    pq = [(0, start)]

    while pq:
        current_dist, node = heapq.heappop(pq)
        if current_dist != distance[node]:
            continue
        for neighbor, weight in graph[node]:
            new_dist = current_dist + weight

            if neighbor not in distance or new_dist < distance[neighbor]:
                distance[neighbor] = new_dist
                heapq.heappush(pq, (new_dist, neighbor))

    return distance
```

Dijkstra: conditions and complexity

Dijkstra: conditions and complexity

Works when

All edge weights are non-negative.

Dijkstra: conditions and complexity

Works when

All edge weights are non-negative.

Does not handle

Negative-weight edges. That requires other algorithms, not covered here.

Dijkstra: conditions and complexity

Works when

All edge weights are non-negative.

Does not handle

Negative-weight edges. That requires other algorithms, not covered here.

Complexity with a heap

$$O((V + E) \log V)$$

Dijkstra: conditions and complexity

Works when

All edge weights are non-negative.

Does not handle

Negative-weight edges. That requires other algorithms, not covered here.

Complexity with a heap

$$O((V + E) \log V)$$

Compared with BFS

BFS uses a normal queue because all priorities are effectively equal by layer.

Dijkstra: conditions and complexity

Works when

All edge weights are non-negative.

Does not handle

Negative-weight edges. That requires other algorithms, not covered here.

Complexity with a heap

$$O((V + E) \log V)$$

Compared with BFS

BFS uses a normal queue because all priorities are effectively equal by layer.

Course rule of thumb: unweighted shortest path means BFS. Weighted shortest path with non-negative weights means Dijkstra.

What to remember

What to remember

1. BFS explores by layers using a queue.

What to remember

1. BFS explores by layers using a queue.
2. BFS gives shortest paths only when every edge has the same cost.

What to remember

1. BFS explores by layers using a queue.
2. BFS gives shortest paths only when every edge has the same cost.
3. Parents reconstruct paths after traversal.

What to remember

1. BFS explores by layers using a queue.
2. BFS gives shortest paths only when every edge has the same cost.
3. Parents reconstruct paths after traversal.
4. DFS explores deeply using recursion or a stack.

What to remember

1. BFS explores by layers using a queue.
2. BFS gives shortest paths only when every edge has the same cost.
3. Parents reconstruct paths after traversal.
4. DFS explores deeply using recursion or a stack.
5. Connected components come from repeated traversal in undirected graphs.

What to remember

1. BFS explores by layers using a queue.
2. BFS gives shortest paths only when every edge has the same cost.
3. Parents reconstruct paths after traversal.
4. DFS explores deeply using recursion or a stack.
5. Connected components come from repeated traversal in undirected graphs.
6. Dijkstra extends shortest-path intuition to non-negative weighted graphs using a priority queue.